# Penetration Testing Report

## Task 4 – Exploitation & Web Application Security Testing

**Prepared By:**
Benidict David

**Organization:**
Maincrafts Technology

**Internship:**
Cybersecurity & Ethical Hacking Internship

# Table of Contents

1. Executive Summary

2. Disclaimer

3. Objective

4. Scope

5. Target Applications

6. Test Environment

7. Tools Used

8. Methodology

9. Finding 1: SQL Injection

10. Finding 2: Reflected Cross Site Scripting

11. Burp Suite Analysis

12. Directory Discovery

13. Risk Assessment

14. Recommendations

15. Learning Outcomes

16. Conclusion

## 1. Executive Summary

This report outlines the findings of a web application penetration test conducted as part of the Cybersecurity & Ethical Hacking Internship at Maincrafts Technology. The assessment focused on identifying and exploiting vulnerabilities within the provided target applications, specifically evaluating susceptibility to SQL Injection, Cross-Site Scripting (XSS), and testing web interception and directory enumeration skills. Successful exploitation demonstrated critical security flaws that require immediate remediation to prevent unauthorized data access and malicious script execution.

## 2. Disclaimer

The penetration testing activities detailed in this report were conducted strictly within a controlled, authorized laboratory environment. The tools and techniques utilized are for educational and authorized testing purposes only. Any application of these techniques outside of an explicitly authorized assessment is illegal and strictly prohibited.

## 3. Objective

The primary objective of this engagement was to perform exploitation and web application security testing to identify vulnerabilities, understand their underlying mechanics, assess their potential business impact, and recommend effective mitigation strategies. This fulfills the requirements for Task 4 of the internship program.

## 4. Scope

The scope of this assessment includes the specific web application instances provided in the local laboratory environment. Testing was restricted to identifying SQL Injection and XSS vulnerabilities, configuring intercepting proxies, and enumerating directories. No external or unauthorized systems were targeted or assessed.

## 5. Target Applications

The following applications were targeted during the assessment:
- Damn Vulnerable Web App (DVWA)
- OWASP Juice Shop

## 6. Test Environment

The testing was performed using a local virtualization setup, utilizing Kali Linux as the primary attack machine. The target applications were hosted locally on the same network to ensure a safe and isolated testing environment.

## 7. Tools Used

The following tools were utilized during the assessment:
• Firefox Web Browser
• FoxyProxy Extension
• Burp Suite Community Edition
• Dirsearch

# 8. Methodology

The assessment followed a structured penetration testing methodology encompassing information gathering, vulnerability identification, and exploitation. This involved manual testing for web vulnerabilities, intercepting and analyzing HTTP traffic using a proxy, and performing automated directory brute-forcing to discover hidden endpoints.

# 9. Finding 1: SQL Injection

## What SQL Injection is:

SQL Injection (SQLi) is a code injection technique where an attacker can execute malicious SQL statements that control a web application's database server. This allows the attacker to view, modify, or delete database records.

OWASP Category: A03:2021-Injection
Risk Level: Critical

## Testing Process and Evidence:

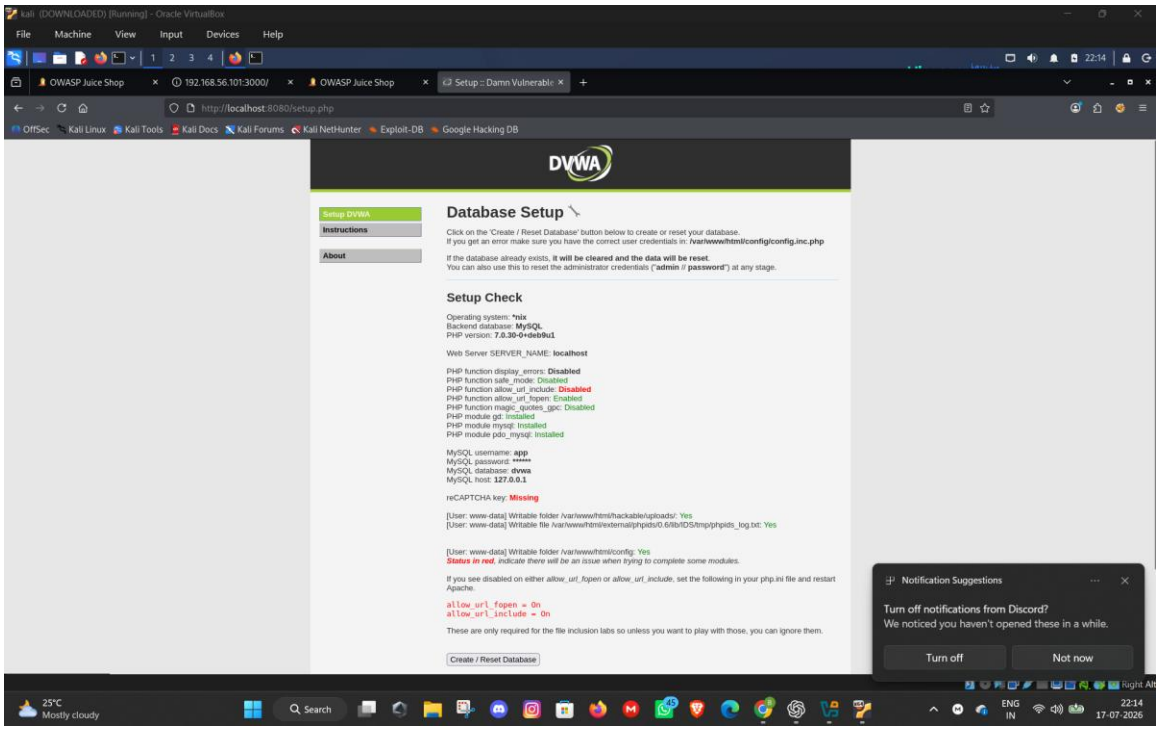

Figure 1 - Title: DVWA Database Setup Screen

Description: Database initialization before testing.
Explanation: The environment was reset and the database initialized to ensure a clean state for testing SQL injection payloads.

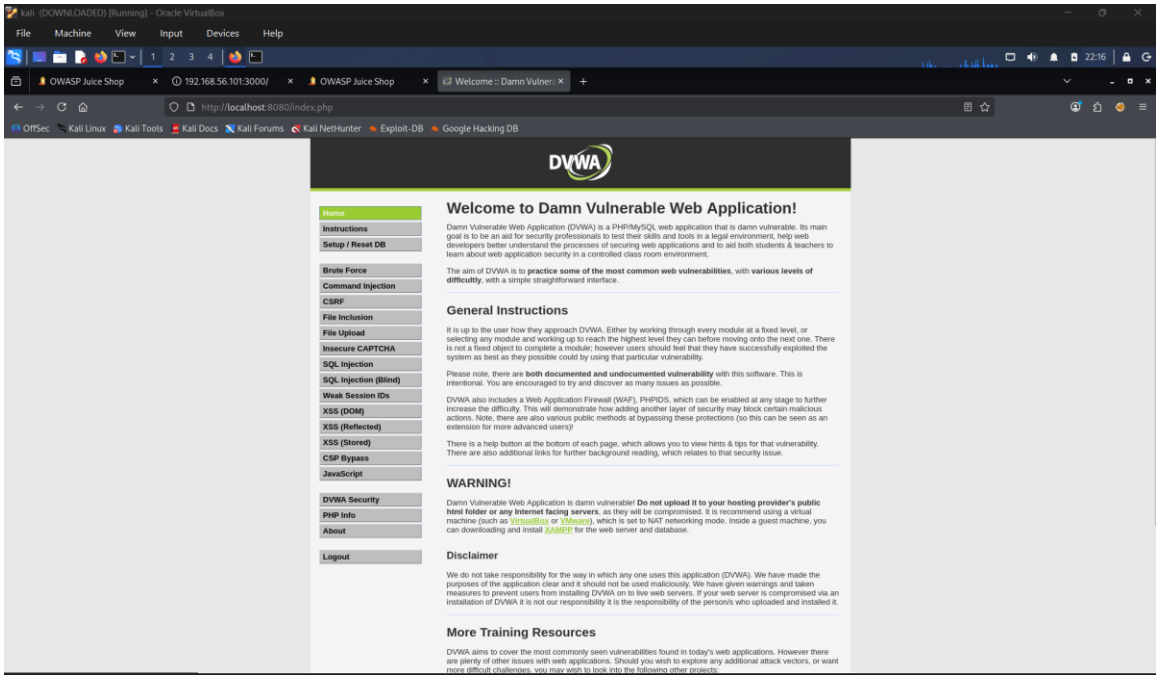


Figure 2 - Title: DVWA Home Page

Description: Successfully configured DVWA environment.
Explanation: Confirms that the target application was properly set up and accessible from the testing machine.

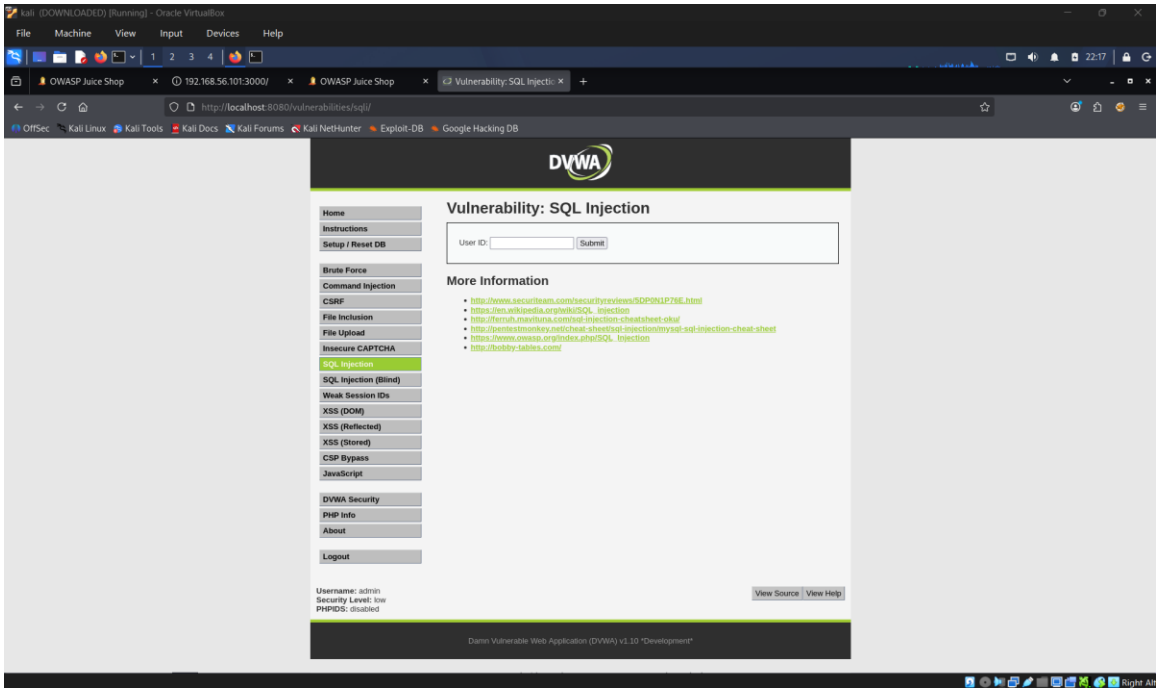


Figure 3 - Title: SQL Injection Module

Description: Target page used for SQL Injection testing.
Explanation: Navigated to the specific module designed to test input validation against database queries.

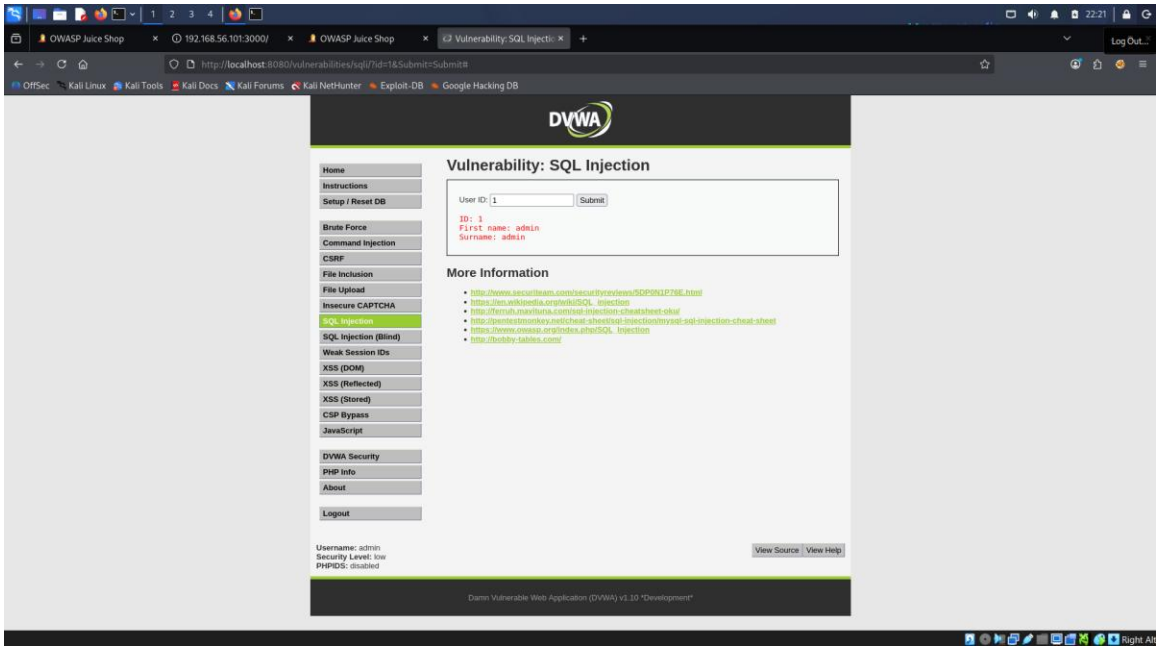


Figure 4 - Title: Normal Query

Description: User ID 1 returned a single database record.
Explanation: Submitting a valid input (1) returned the expected legitimate database entry, establishing the baseline behavior of the application.

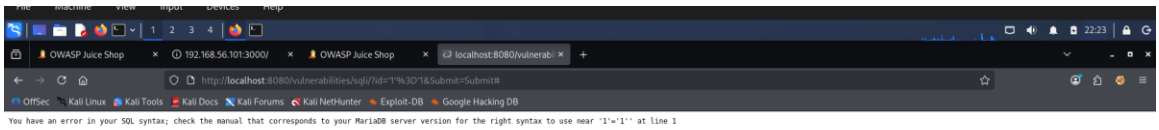


Figure 5 - Title: Failed SQL Injection

Description: Initial payload caused a SQL syntax error before payload refinement.
Explanation: An initial test using a single quote (') resulted in a database error, confirming that user input is being directly concatenated into the SQL query, indicating a vulnerability.

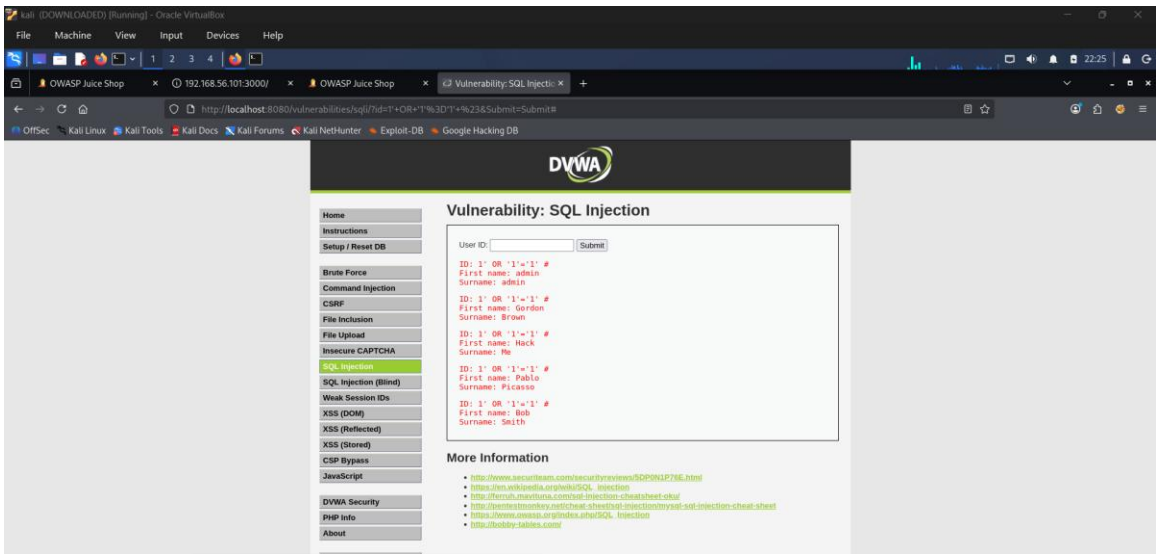


Figure 6 - Title: Successful SQL Injection

Description: Payload 1' OR '1'='1' # returned multiple user records.
Explanation: The successful payload 1' OR '1'='1' # was injected. The 1' closes the original string, the OR '1'='1' condition always evaluates to true, and the # comments out the rest of the original query. This manipulation caused the database to return all user records instead of just one, bypassing authentication or data restriction mechanisms.

## Business Impact:

A successful SQL injection attack can lead to complete database compromise, resulting in the theft of sensitive customer data, intellectual property, and user credentials. It can also

allow an attacker to modify or delete data, causing severe reputational damage and regulatory fines.

### Mitigation:

Implement Prepared Statements (Parameterized Queries) to ensure that the database treats user input as data, not as executable code. Additionally, enforce strict Input Validation and utilize an Object-Relational Mapping (ORM) framework.

## 10. Finding 2: Reflected Cross Site Scripting

### What Reflected XSS is:

Reflected Cross-Site Scripting (XSS) occurs when malicious script injected by an attacker is reflected off a web server, such as in an error message or search result. The payload is not stored on the server but is executed by the victim's browser when they click a crafted link.

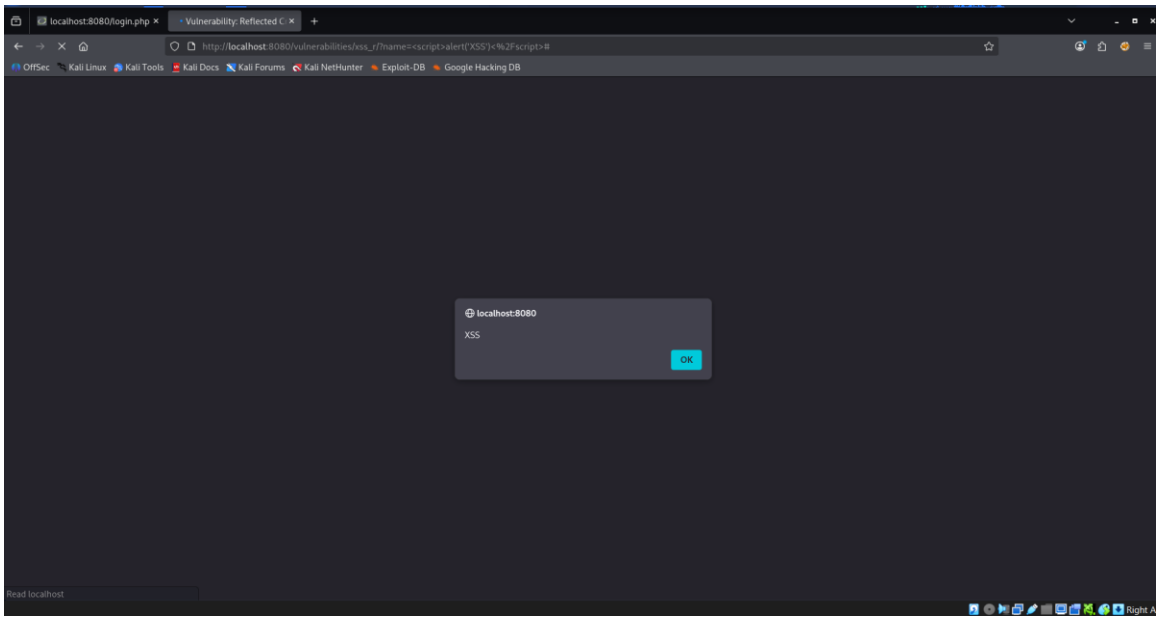


Figure 7 - Title: Successful Reflected Cross Site Scripting

Description: Browser executed injected JavaScript.
Explanation: A crafted JavaScript payload was entered into an input field. The application reflected this payload back to the browser without proper encoding, causing the browser to

execute the JavaScript and display an alert box. This demonstrates that an attacker can run arbitrary scripts within the context of the user's session.

### Business Impact:

XSS vulnerabilities allow attackers to hijack user sessions, steal sensitive cookies, redirect users to malicious sites, or deface websites. This can lead to account takeover and loss of user trust.

### Mitigation:

Implement strict Output Encoding (context-aware escaping) to ensure that any user-supplied data rendered in the browser is treated as text, not executable code. Additionally, deploy a Content Security Policy (CSP) to restrict the sources from which scripts can be loaded.

## 11. Burp Suite Analysis

### Burp Suite Setup and Proxy Configuration:

Burp Suite acts as an intercepting proxy, sitting between the browser and the web application.

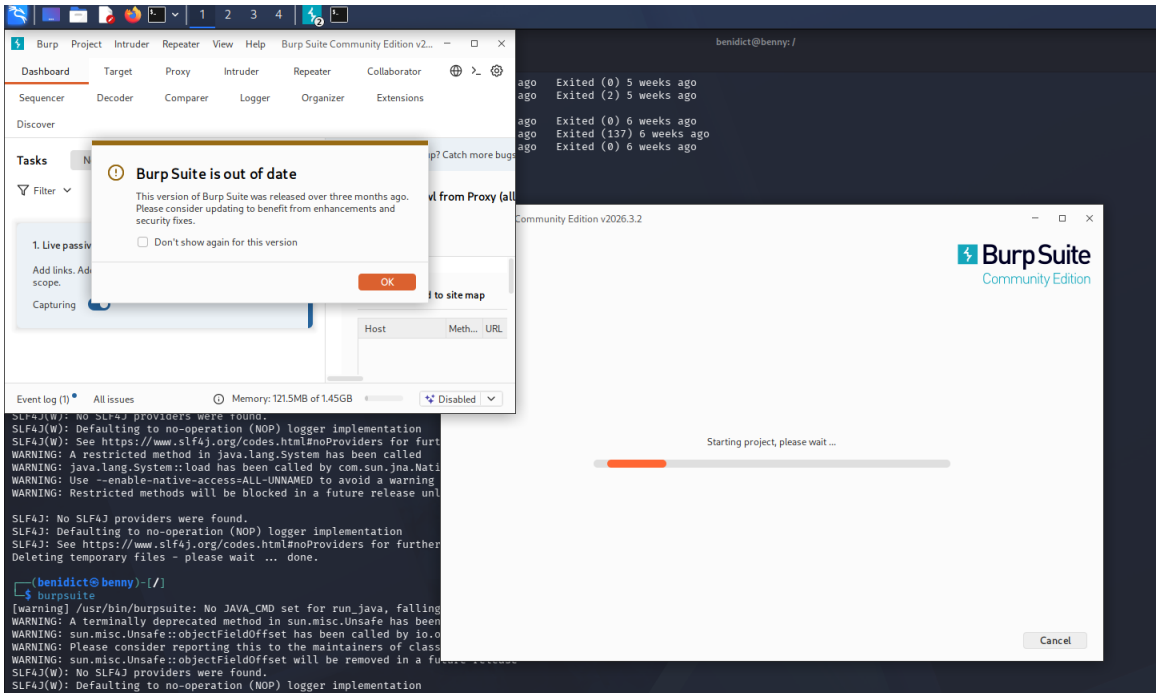


Figure 8 - Title: Burp Suite Startup

Description: Launching the Burp Suite Community Edition application.
Explanation: Burp Suite was initialized to prepare for traffic interception.

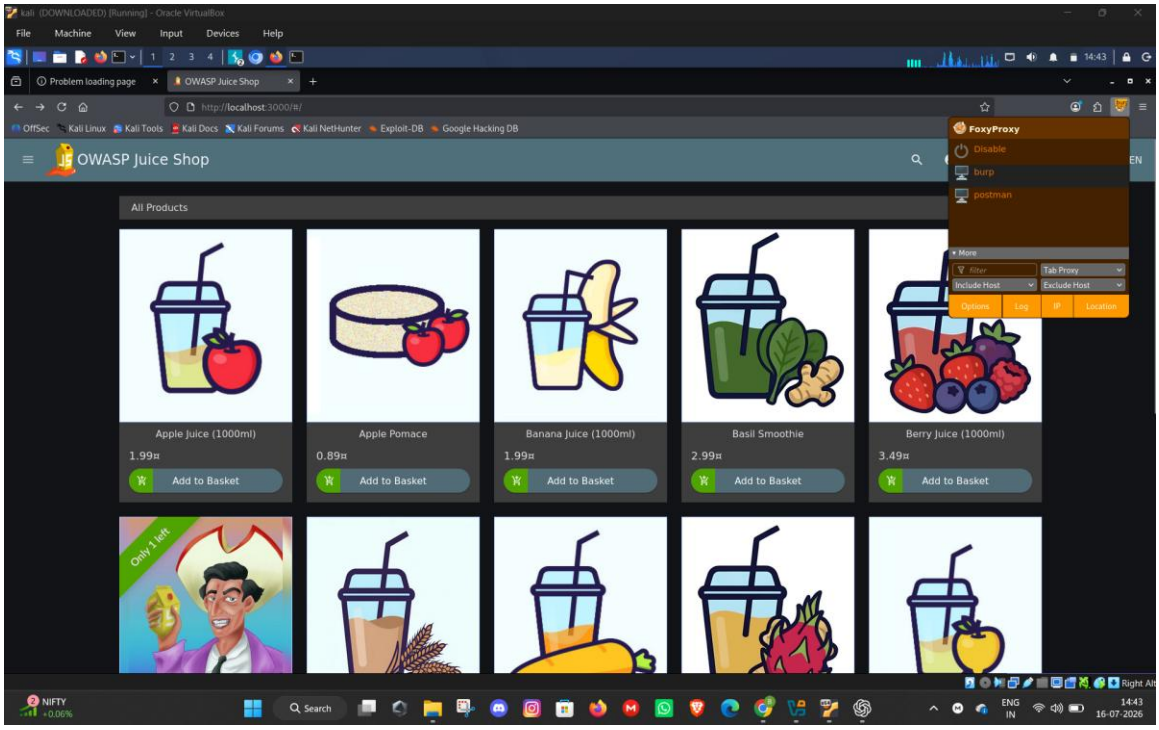


Figure 9 - Title: FoxyProxy Enabled

Description: Configuring the browser to route traffic through Burp Suite.
Explanation: The FoxyProxy extension was enabled in Firefox, routing all web traffic to localhost on port 8080, where Burp Suite is listening.

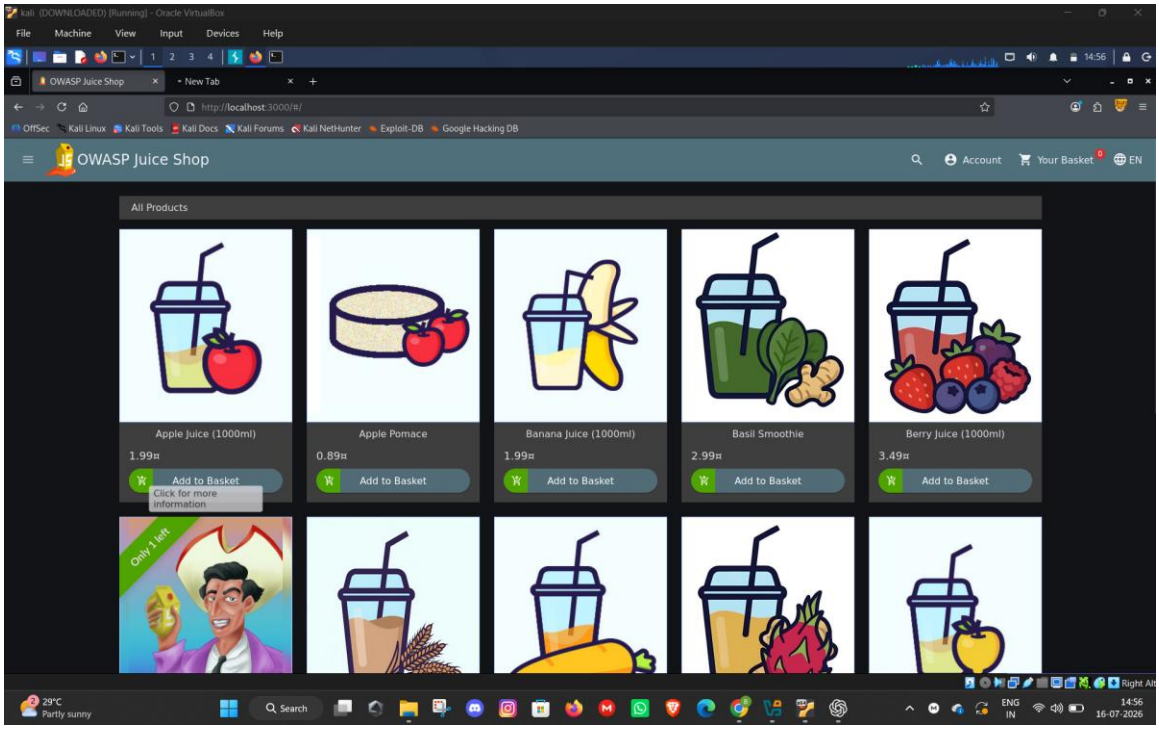

Figure 12 - Title: OWASP Juice Shop Homepage

Description: Target application used for Burp Suite interception and directory enumeration.
Explanation: The target application was accessed through the proxied browser to generate traffic.

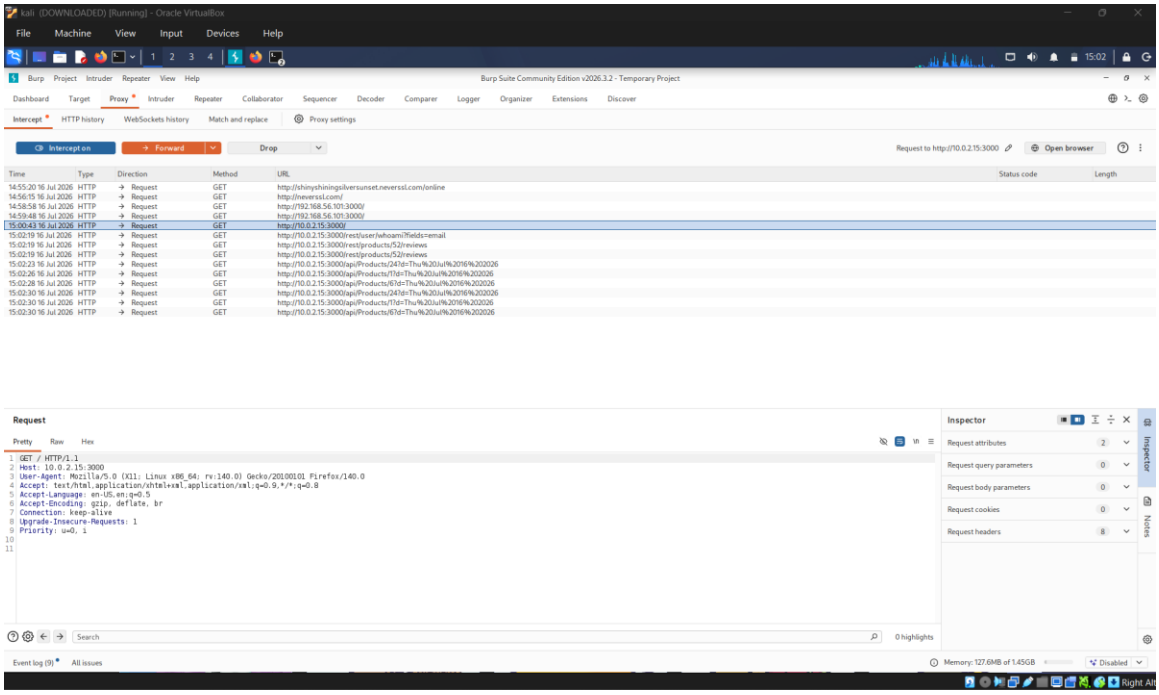

Figure 10 - Title: Burp HTTP History

Description: Captured requests between Firefox and OWASP Juice Shop.
Explanation: The HTTP History tab in Burp Suite successfully captured the GET requests made to the target application. This demonstrates the ability to intercept, view, and potentially modify the traffic in transit.

## Why Burp is important during penetration testing:

Burp Suite is essential because it provides granular visibility into the HTTP communication between the client and server. It allows testers to manipulate parameters, headers, and cookies before they reach the server, enabling the discovery of vulnerabilities that cannot be found through simple browser interaction.

# 12. Directory Discovery

## Directory Enumeration with Dirsearch:

Directory enumeration is the process of using automated tools to discover hidden files and directories on a web server that are not publicly linked. This is often achieved by brute-forcing potential paths using a wordlist.


Figure 11 - Title: Directory Enumeration

Description: Discovered endpoints using Dirsearch.
Explanation: The tool Dirsearch was executed against the target application. It successfully discovered several endpoints that returned HTTP 200 and 301 status codes, including /assets, /media, /ftp, /promotion, /robots.txt, and /video. These discovered paths could reveal sensitive information or administrative interfaces that were intended to be hidden.

## 13. Risk Assessment

| Vulnerability | Likelihood | Impact | Overall Risk |
|---|---|---|---|
| SQL Injection | High | Critical | Critical |
| Reflected XSS | Medium | High | High |
| Information Disclosure via Hidden Directories | High | Medium | Medium |

## 14. Recommendations

To improve the security posture of the applications, the following recommendations should be implemented:

- **Prepared Statements & Parameterized Queries:** Use these for all database interactions to prevent SQL Injection.
- **Input Validation:** Implement strict allow-lists for all user inputs on both client and server sides.
- **Output Encoding:** Contextually encode all user-supplied data before rendering it in the browser to prevent XSS.
- **Content Security Policy (CSP):** Implement a robust CSP to restrict the execution of unauthorized scripts.
- **Least Privilege:** Ensure the database user account used by the application has only the minimum necessary permissions.
- **Web Application Firewall (WAF):** Deploy a WAF to detect and block common attack payloads.
- **Regular Security Testing:** Conduct periodic vulnerability scans and penetration tests to identify and address new flaws.

## 15. Learning Outcomes

This task provided hands-on experience with fundamental web application penetration testing techniques. Key skills acquired include identifying and exploiting SQL Injection to bypass access controls, demonstrating the impact of Reflected XSS, configuring and utilizing intercepting proxies like Burp Suite for traffic analysis, and performing automated directory enumeration to map an application's attack surface.

## 16. Conclusion

The penetration testing activities performed during this task successfully demonstrated the presence of critical vulnerabilities within the target applications. The exploitation of SQL Injection and Reflected XSS in a controlled laboratory environment highlights the severe risks associated with improper input handling and output rendering. Implementing the recommended remediation strategies, such as parameterized queries and output encoding, is essential to secure these applications against malicious attacks.